

CIS 6270

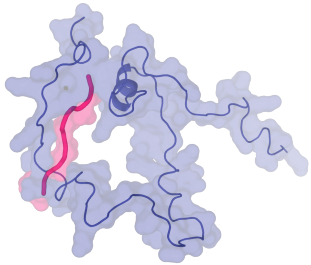
Module 6: Flow Maps

Lecture 6.2: Consistency, Distillation, and Discrete Flow Maps

Thu, Nov 12, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Let's read a Jacobian numerically

Let's read a Jacobian numerically

In one dimension, a Jacobian $J_{s,t}(x) = 3$ means that a small starting displacement of 0.01 produces an approximate destination displacement of 0.03.

Let's read a Jacobian numerically

In one dimension, a Jacobian $J_{s,t}(x) = 3$ means that a small starting displacement of 0.01 produces an approximate destination displacement of 0.03.

In two dimensions, stretching one coordinate by 2 and the other by 3 multiplies a small area by 6.

Let's read a Jacobian numerically

In one dimension, a Jacobian $J_{s,t}(x) = 3$ means that a small starting displacement of 0.01 produces an approximate destination displacement of 0.03.

In two dimensions, stretching one coordinate by 2 and the other by 3 multiplies a small area by 6. The determinant det J is this area multiplier.

How does the Jacobian change as we follow the trajectory?

How does the Jacobian change as we follow the trajectory?

$$\boxed{\frac{d}{dt} J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

How does the Jacobian change as we follow the trajectory?

$$\boxed{\frac{d}{dt} J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

The Jacobian $J_{s,t}(x)$ changes at a rate equal to the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ times $J_{s,t}(x)$.

How does the Jacobian change as we follow the trajectory?

$$\boxed{\frac{d}{dt} J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

The Jacobian $J_{s,t}(x)$ changes at a rate equal to the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ times $J_{s,t}(x)$. At the starting time, $J_{s,s}(x)$ is the identity matrix I.

How does the Jacobian change as we follow the trajectory?

$$\boxed{\frac{d}{dt}J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

The Jacobian $J_{s,t}(x)$ changes at a rate equal to the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ times $J_{s,t}(x)$. At the starting time, $J_{s,s}(x)$ is the identity matrix I.

Differentiate the flow equation in x and exchange the derivatives in x and t.

How does the Jacobian change as we follow the trajectory?

$$\boxed{\frac{d}{dt} J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

The Jacobian $J_{s,t}(x)$ changes at a rate equal to the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ times $J_{s,t}(x)$. At the starting time, $J_{s,s}(x)$ is the identity matrix I.

Differentiate the flow equation in x and exchange the derivatives in x and t. The chain rule gives the velocity gradient ∇b at the destination $X_{s,t}(x)$.

How does the Jacobian change as we follow the trajectory?

$$\boxed{\frac{d}{dt} J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

The Jacobian $J_{s,t}(x)$ changes at a rate equal to the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ times $J_{s,t}(x)$. At the starting time, $J_{s,s}(x)$ is the identity matrix I.

Differentiate the flow equation in x and exchange the derivatives in x and t. The chain rule gives the velocity gradient ∇b at the destination $X_{s,t}(x)$.

This equation is linear in the Jacobian $J_{s,t}(x)$.

How does the Jacobian change as we follow the trajectory?

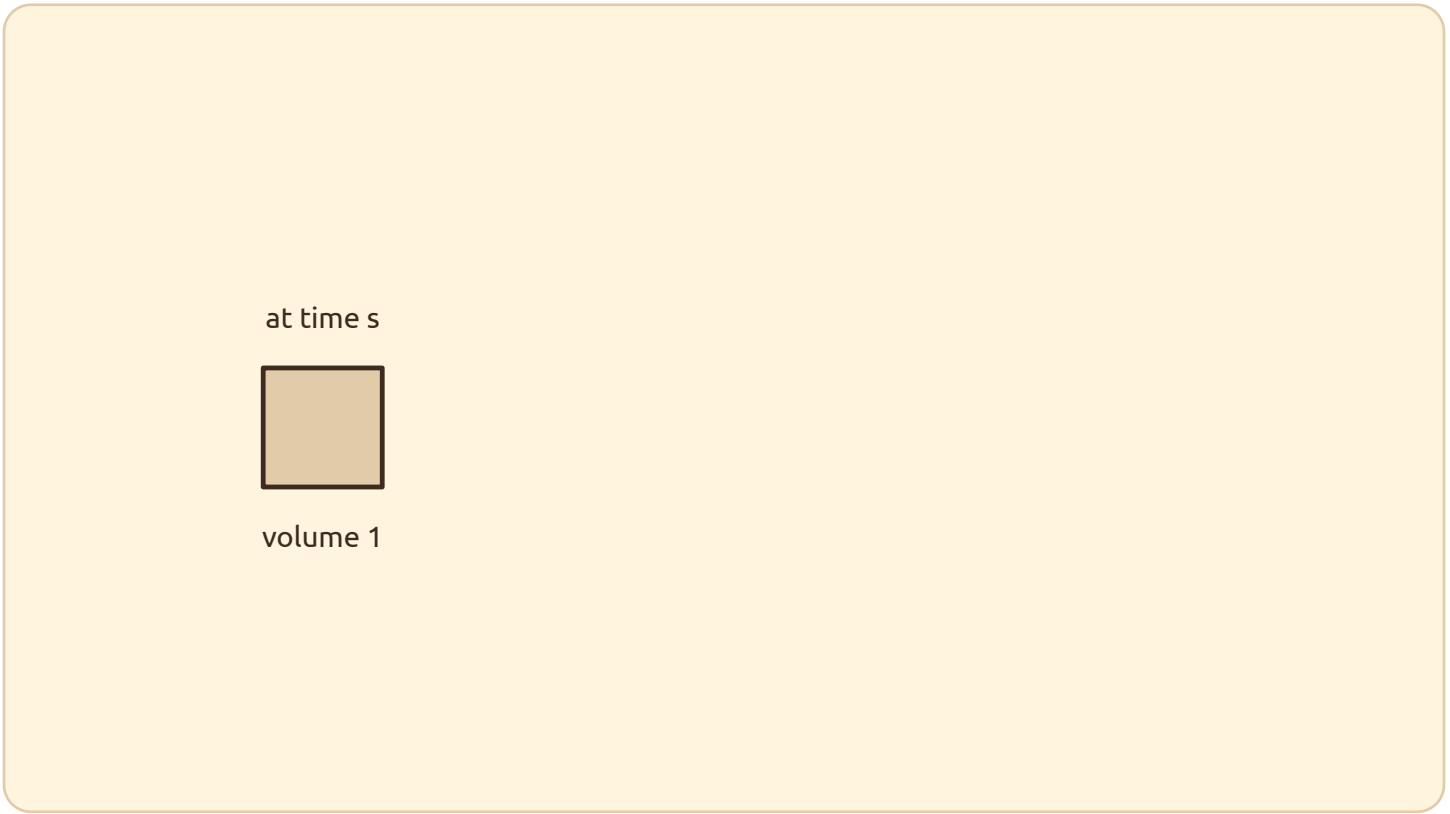
$$\boxed{\frac{d}{dt} J_{s,t}(x) = \nabla b\big(X_{s,t}(x),\, t\big)\, J_{s,t}(x), \qquad J_{s,s}(x) = I}$$

The Jacobian $J_{s,t}(x)$ changes at a rate equal to the velocity-gradient matrix $\nabla b(X_{s,t}(x), t)$ times $J_{s,t}(x)$. At the starting time, $J_{s,s}(x)$ is the identity matrix I.

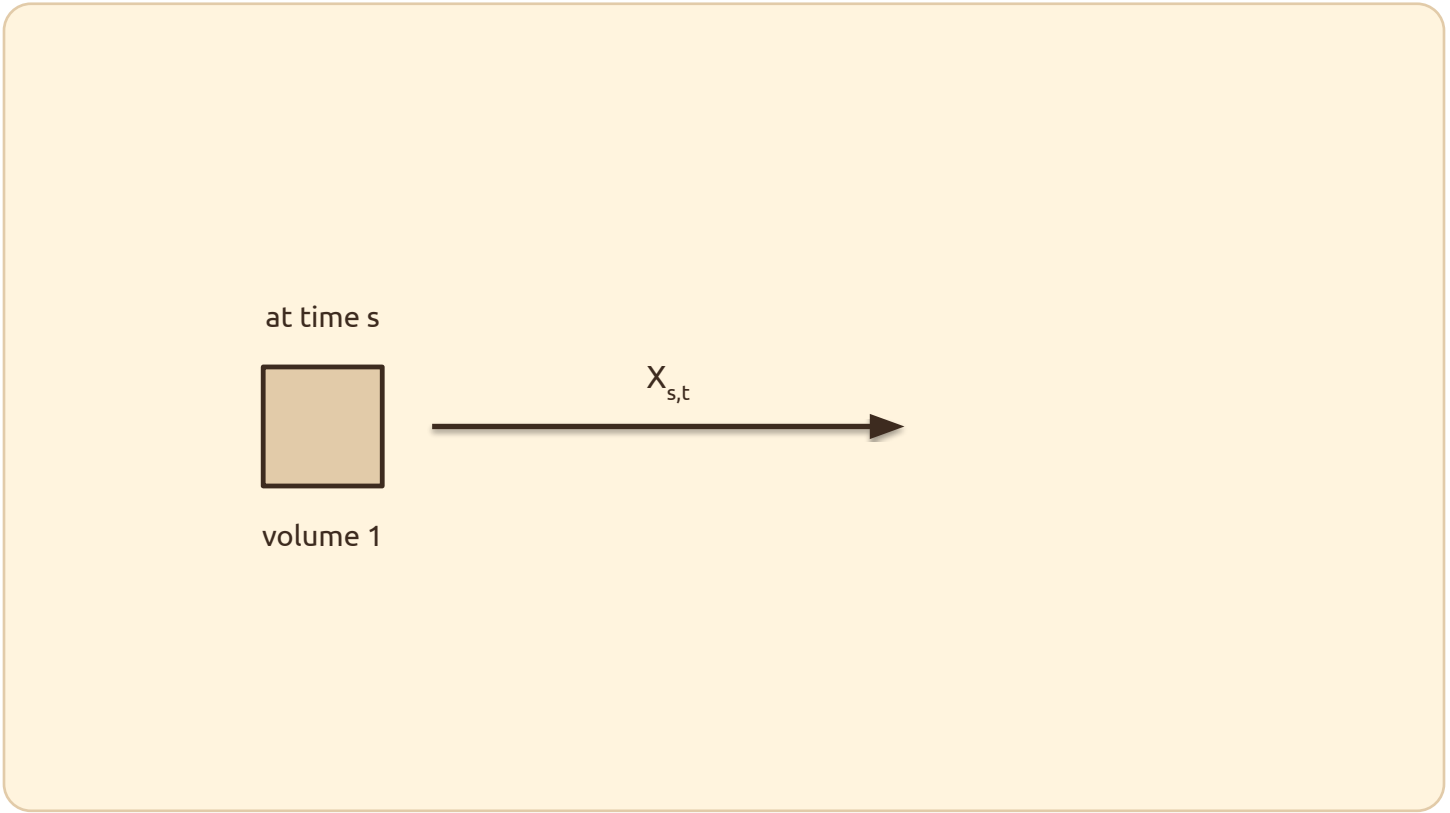
Differentiate the flow equation in x and exchange the derivatives in x and t. The chain rule gives the velocity gradient ∇b at the destination $X_{s,t}(x)$.

This equation is linear in the Jacobian $J_{s,t}(x)$. We evaluate the velocity gradient ∇b along the fixed trajectory, even when b is nonlinear in x.

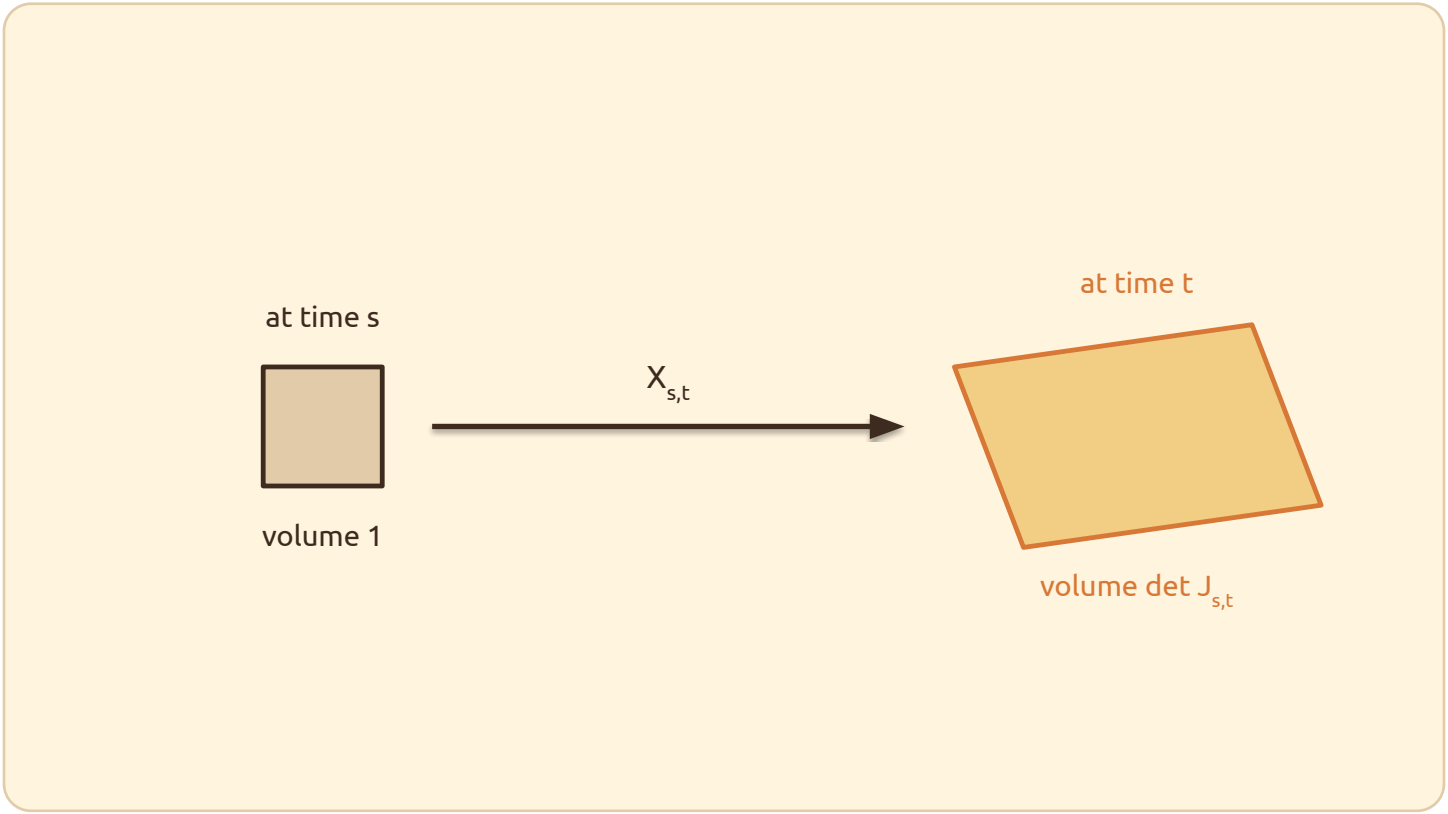
Take a small box of volume 1 at time s



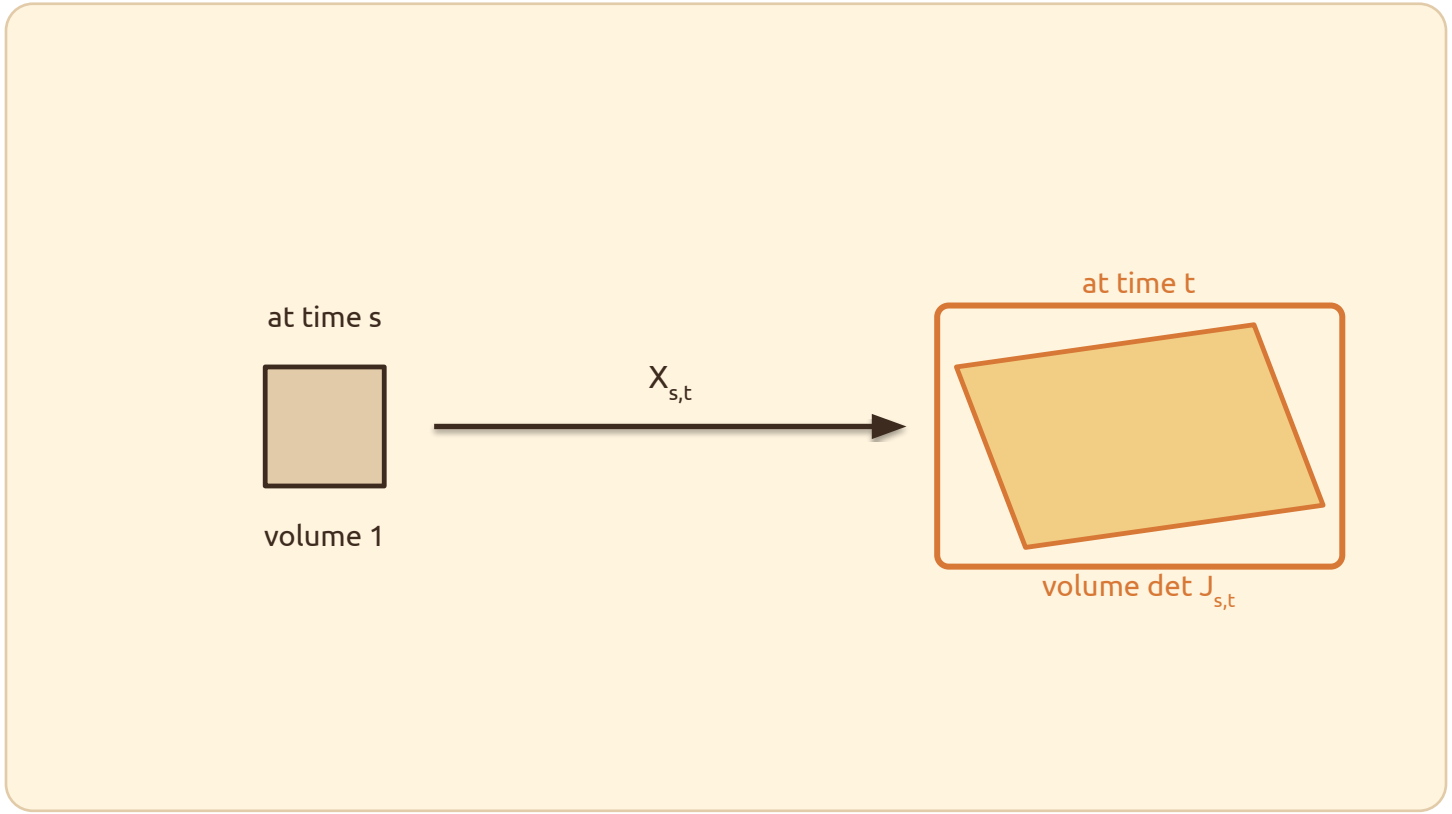
Carry the box along the flow



It arrives stretched and sheared



Its volume is multiplied by the determinant of the Jacobian



How does local volume change along the flow?

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$.

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$. This is Liouville's formula.

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$. This is Liouville's formula.

Liouville's formula relates the local expansion of the flow to the volume change of a small region.

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$. This is Liouville's formula.

For an invertible matrix J, the derivative of log det J is the trace of J^{-1} times the derivative of J.

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$. This is Liouville's formula.

For an invertible matrix J, the derivative of log det J is the trace of J^{-1} times the derivative of J. The trace is the sum of the diagonal entries.

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$. This is Liouville's formula.

For an invertible matrix J, the derivative of log det J is the trace of J^{-1} times the derivative of J. The trace is the sum of the diagonal entries.

Substituting the Jacobian equation leaves the trace of ∇b, which is the divergence $\nabla \cdot b$.

How does local volume change along the flow?

$$\boxed{\frac{d}{dt}\log\det J_{s,t}(x) = \nabla \cdot b\big(X_{s,t}(x),\ t\big)}$$

The log volume scale log det $J_{s,t}(x)$ changes at the rate $\nabla \cdot b(X_{s,t}(x), t)$, the velocity divergence at the moving point $X_{s,t}(x)$. This is Liouville's formula.

For an invertible matrix J, the derivative of log det J is the trace of J^{-1} times the derivative of J. The trace is the sum of the diagonal entries.

Substituting the Jacobian equation leaves the trace of ∇b, which is the divergence $\nabla \cdot b$.

When the divergence $\nabla \cdot b$ is zero, a small region keeps its volume as it moves.

Let's turn divergence into a volume change

Let's turn divergence into a volume change

Suppose the velocity divergence $\nabla \cdot b$ stays at log 2 for one time unit.

Let's turn divergence into a volume change

Suppose the velocity divergence $\nabla \cdot b$ stays at log 2 for one time unit. Liouville's formula gives log det J = log 2, so det J = 2.

Let's turn divergence into a volume change

Suppose the velocity divergence $\nabla \cdot b$ stays at log 2 for one time unit. Liouville's formula gives log det J = log 2, so det J = 2.

A small region doubles in volume.

Let's turn divergence into a volume change

Suppose the velocity divergence $\nabla \cdot b$ stays at log 2 for one time unit. Liouville's formula gives log det J = log 2, so det J = 2.

A small region doubles in volume. Since it contains the same probability mass, its density falls by a factor of two.

Let's calculate volume changes in a Gaussian example

Let's calculate volume changes in a Gaussian example

Take the source N(0, 1) and the data N(2, 9), paired by $x_1 = 2 + 3x_0$, and interpolate linearly.

Let's calculate volume changes in a Gaussian example

Take the source N(0, 1) and the data N(2, 9), paired by $x_1 = 2 + 3x_0$, and interpolate linearly.

The map and the velocity are

$$X_{0,t}(x) = 2t + (1 + 2t)\, x, \qquad b(x, t) = 2 + \frac{2\,(x - 2t)}{1 + 2t}$$

Let's calculate volume changes in a Gaussian example

Take the source N(0, 1) and the data N(2, 9), paired by $x_1 = 2 + 3x_0$, and interpolate linearly.

The map and the velocity are

$$X_{0,t}(x) = 2t + (1 + 2t)\, x, \qquad b(x, t) = 2 + \frac{2\,(x - 2t)}{1 + 2t}$$

The flow map $X_{0,t}(x)$ shifts the starting state x by 2t and scales it by 1 + 2t.

Let's calculate volume changes in a Gaussian example

Take the source N(0, 1) and the data N(2, 9), paired by $x_1 = 2 + 3x_0$, and interpolate linearly.

The map and the velocity are

$$X_{0,t}(x) = 2t + (1 + 2t)\, x, \qquad b(x, t) = 2 + \frac{2\,(x - 2t)}{1 + 2t}$$

The flow map $X_{0,t}(x)$ shifts the starting state x by 2t and scales it by 1 + 2t. The instantaneous velocity b(x, t) is 2 plus 2(x – 2t) divided by 1 + 2t.

Let's calculate volume changes in a Gaussian example

Take the source N(0, 1) and the data N(2, 9), paired by $x_1 = 2 + 3x_0$, and interpolate linearly.

The map and the velocity are

$$X_{0,t}(x) = 2t + (1 + 2t)\, x, \qquad b(x, t) = 2 + \frac{2\,(x - 2t)}{1 + 2t}$$

The flow map $X_{0,t}(x)$ shifts the starting state x by 2t and scales it by 1 + 2t. The instantaneous velocity b(x, t) is 2 plus 2(x – 2t) divided by 1 + 2t.

The term 2t shifts the mean, while the factor 1 + 2t scales the spread.

Compute the Jacobian and the divergence of this flow

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

The Jacobian $J_{0,t}$ is 1 + 2t.

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

The Jacobian $J_{0,t}$ is 1 + 2t. The velocity divergence $\nabla \cdot b$ is 2 divided by 1 + 2t.

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

The Jacobian $J_{0,t}$ is 1 + 2t. The velocity divergence $\nabla \cdot b$ is 2 divided by 1 + 2t. Its integral from time 0 to 1 is log 3.

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

The Jacobian $J_{0,t}$ is 1 + 2t. The velocity divergence $\nabla \cdot b$ is 2 divided by 1 + 2t. Its integral from time 0 to 1 is log 3.

At t = 1, the Jacobian $J_{0,1}$ is 3.

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

The Jacobian $J_{0,t}$ is 1 + 2t. The velocity divergence $\nabla \cdot b$ is 2 divided by 1 + 2t. Its integral from time 0 to 1 is log 3.

At t = 1, the Jacobian $J_{0,1}$ is 3. Liouville's formula gives log det $J_{0,1}$ = log 3.

Compute the Jacobian and the divergence of this flow

$$J_{0,t} = 1 + 2t, \qquad \nabla \cdot b = \frac{2}{1+2t}, \qquad \int_0^1 \frac{2\,dt}{1+2t} = \log 3$$

The Jacobian $J_{0,t}$ is 1 + 2t. The velocity divergence $\nabla \cdot b$ is 2 divided by 1 + 2t. Its integral from time 0 to 1 is log 3.

At t = 1, the Jacobian $J_{0,1}$ is 3. Liouville's formula gives log det $J_{0,1}$ = log 3.

The flow stretches every length by 3, so it divides every density by 3.

Now transport a probability density along the flow

Now transport a probability density along the flow

Probability is conserved along the flow:

$$\boxed{p_t\big(X_{s,t}(x)\big)\ \big|\det J_{s,t}(x)\big| \ = \ p_s(x)}$$

Now transport a probability density along the flow

Probability is conserved along the flow: $$\boxed{p_t\big(X_{s,t}(x)\big)\ \big|\det J_{s,t}(x)\big| \ = \ p_s(x)}$$

The transported density $p_t(X_{s,t}(x))$, multiplied by the local volume scale $|\det J_{s,t}(x)|$, equals the starting density $p_s(x)$.

Now transport a probability density along the flow

Probability is conserved along the flow: $$\boxed{p_t\big(X_{s,t}(x)\big)\ \big|\det J_{s,t}(x)\big| \;=\; p_s(x)}$$

The transported density $p_t(X_{s,t}(x))$, multiplied by the local volume scale $|\det J_{s,t}(x)|$, equals the starting density $p_s(x)$.

The total mass in a box of probability stays the same, so where the flow stretches the box the density falls by the same factor.

Now transport a probability density along the flow

Probability is conserved along the flow: $$\boxed{p_t\big(X_{s,t}(x)\big)\ \big|\det J_{s,t}(x)\big| \;=\; p_s(x)}$$

The transported density $p_t(X_{s,t}(x))$, multiplied by the local volume scale $|\det J_{s,t}(x)|$, equals the starting density $p_s(x)$.

The total mass in a box of probability stays the same, so where the flow stretches the box the density falls by the same factor.

In the Gaussian example, the volume scale $J_{0,1}$ is 3.

Now transport a probability density along the flow

Probability is conserved along the flow: $$\boxed{p_t\big(X_{s,t}(x)\big)\ \big|\det J_{s,t}(x)\big| \ = \ p_s(x)}$$

The transported density $p_t(X_{s,t}(x))$, multiplied by the local volume scale $|\det J_{s,t}(x)|$, equals the starting density $p_s(x)$.

The total mass in a box of probability stays the same, so where the flow stretches the box the density falls by the same factor.

In the Gaussian example, the volume scale $J_{0,1}$ is 3. The final density $p_1(2 + 3x)$ is therefore one third of the starting density $p_0(x)$.

Let's turn that statement into a differential equation

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

The transported log density log $p_t(X_{s,t}(x))$ plus the log volume scale log det $J_{s,t}(x)$ equals the starting log density log $p_s(x)$, which is constant in t.

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$

$$\boxed{\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0}$$

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

$$\boxed{\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0}$$

The log-density change $\partial_t \log p_t$, the transport term $b \cdot \nabla \log p_t$, and the velocity divergence $\nabla \cdot b$ sum to zero.

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

$$\boxed{\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0}$$

The log-density change $\partial_t \log p_t$, the transport term $b \cdot \nabla \log p_t$, and the velocity divergence $\nabla \cdot b$ sum to zero. Equivalently, the density change $\partial_t p_t$ plus the probability-flux divergence $\nabla \cdot (p_t b)$ is zero.

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

$$\boxed{\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0}$$

The log-density change $\partial_t \log p_t$, the transport term $b \cdot \nabla \log p_t$, and the velocity divergence $\nabla \cdot b$ sum to zero. Equivalently, the density change $\partial_t p_t$ plus the probability-flux divergence $\nabla \cdot (p_t b)$ is zero.

We obtain the first two terms by applying the chain rule to the log density.

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

$$\boxed{\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0}$$

The log-density change $\partial_t \log p_t$, the transport term $b \cdot \nabla \log p_t$, and the velocity divergence $\nabla \cdot b$ sum to zero. Equivalently, the density change $\partial_t p_t$ plus the probability-flux divergence $\nabla \cdot (p_t b)$ is zero.

We obtain the first two terms by applying the chain rule to the log density. The third term follows from Liouville's equation.

Let's turn that statement into a differential equation

Take logarithms and differentiate in t: $$\log p_t\big(X_{s,t}(x)\big) + \log\det J_{s,t}(x) = \log p_s(x)$$

$$\boxed{\partial_t \log p_t + b \cdot \nabla \log p_t + \nabla \cdot b = 0 \quad \Longleftrightarrow \quad \partial_t p_t + \nabla \cdot (p_t\, b) = 0}$$

The log-density change $\partial_t \log p_t$, the transport term $b \cdot \nabla \log p_t$, and the velocity divergence $\nabla \cdot b$ sum to zero. Equivalently, the density change $\partial_t p_t$ plus the probability-flux divergence $\nabla \cdot (p_t b)$ is zero.

We obtain the first two terms by applying the chain rule to the log density. The third term follows from Liouville's equation.

We have recovered the continuity equation used in continuous flow matching directly from the flow map.

Now we can write the likelihood of a generated sample

Now we can write the likelihood of a generated sample

$$\log p_1\big(X_{0,1}(x_0)\big) = \log p_0(x_0) - \int_0^1 \nabla \cdot b\big(X_{0,t}(x_0),\ t\big)\, dt$$

Now we can write the likelihood of a generated sample

$$\log p_1\big(X_{0,1}(x_0)\big) = \log p_0(x_0) - \int_0^1 \nabla \cdot b\big(X_{0,t}(x_0),\ t\big)\, dt$$

The generated log density $\log p_1(X_{0,1}(x_0))$ equals the source log density $\log p_0(x_0)$ minus the time integral of the velocity divergence $\nabla \cdot b(X_{0,t}(x_0), t)$ along the trajectory.

Now we can write the likelihood of a generated sample

$$\log p_1\big(X_{0,1}(x_0)\big) = \log p_0(x_0) - \int_0^1 \nabla \cdot b\big(X_{0,t}(x_0),\ t\big)\, dt$$

The generated log density log $p_1(X_{0,1}(x_0))$ equals the source log density log $p_0(x_0)$ minus the time integral of the velocity divergence $\nabla \cdot b(X_{0,t}(x_0), t)$ along the trajectory.

Subtract the integrated divergence from the source log density.

Now we can write the likelihood of a generated sample

$$\log p_1\big(X_{0,1}(x_0)\big) = \log p_0(x_0) - \int_0^1 \nabla \cdot b\big(X_{0,t}(x_0),\ t\big)\, dt$$

The generated log density log $p_1(X_{0,1}(x_0))$ equals the source log density log $p_0(x_0)$ minus the time integral of the velocity divergence $\nabla \cdot b(X_{0,t}(x_0), t)$ along the trajectory.

Subtract the integrated divergence from the source log density.

We can therefore use a continuous normalizing flow to evaluate likelihoods.

Now we can write the likelihood of a generated sample

$$\log p_1\big(X_{0,1}(x_0)\big) = \log p_0(x_0) - \int_0^1 \nabla \cdot b\big(X_{0,t}(x_0),\ t\big)\, dt$$

The generated log density log $p_1(X_{0,1}(x_0))$ equals the source log density log $p_0(x_0)$ minus the time integral of the velocity divergence $\nabla \cdot b(X_{0,t}(x_0), t)$ along the trajectory.

Subtract the integrated divergence from the source log density.

We can therefore use a continuous normalizing flow to evaluate likelihoods. Computing the integral takes many evaluations.

How quickly does a measurement change along the flow?

How quickly does a measurement change along the flow?

Let f be a measurement:

$$\boxed{(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)}$$

How quickly does a measurement change along the flow?

Let f be a measurement:

$$\boxed{(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)}$$

The generator $\mathcal{L}_t$ gives the instantaneous change in a measurement f.

How quickly does a measurement change along the flow?

Let f be a measurement: $$\boxed{(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)}$$

The generator $\mathcal{L}_t$ gives the instantaneous change in a measurement f. Its value $(\mathcal{L}_t f)(x)$ is the velocity b(x, t) dotted with the measurement gradient $\nabla f(x)$.

How quickly does a measurement change along the flow?

Let f be a measurement: $$\boxed{(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)}$$

The generator $\mathcal{L}_t$ gives the instantaneous change in a measurement f. Its value $(\mathcal{L}_t f)(x)$ is the velocity b(x, t) dotted with the measurement gradient $\nabla f(x)$.

For example, if f measures one coordinate, then $\mathcal{L}_t f$ is the velocity in that coordinate.

How quickly does a measurement change along the flow?

Let f be a measurement: $$\boxed{(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)}$$

The generator $\mathcal{L}_t$ gives the instantaneous change in a measurement f. Its value $(\mathcal{L}_t f)(x)$ is the velocity b(x, t) dotted with the measurement gradient $\nabla f(x)$.

For example, if f measures one coordinate, then $\mathcal{L}_t f$ is the velocity in that coordinate.

So for any f and any trajectory $$\frac{d}{dt} f\big(X_{s,t}(x)\big) = \big(\mathcal{L}_t f\big)\big(X_{s,t}(x)\big)$$

How quickly does a measurement change along the flow?

Let f be a measurement: $$\boxed{(\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x)}$$

The generator $\mathcal{L}_t$ gives the instantaneous change in a measurement f. Its value $(\mathcal{L}_t f)(x)$ is the velocity b(x, t) dotted with the measurement gradient $\nabla f(x)$.

For example, if f measures one coordinate, then $\mathcal{L}_t f$ is the velocity in that coordinate.

So for any f and any trajectory $$\frac{d}{dt} f\big(X_{s,t}(x)\big) = \big(\mathcal{L}_t f\big)\big(X_{s,t}(x)\big)$$

The measurement $f(X_{s,t}(x))$ changes at the rate $(\mathcal{L}_t f)(X_{s,t}(x))$, the generator applied to f at the moving point $X_{s,t}(x)$.

Let's calculate a generator with numbers

Let's calculate a generator with numbers

Let the measurement be $f(x) = x^2$.

Let's calculate a generator with numbers

Let the measurement be $f(x) = x^2$. At state $x = 2$ with velocity $b(x, t) = 3$, its spatial derivative is $f'(2) = 4$.

Let's calculate a generator with numbers

Let the measurement be $f(x) = x^2$. At state $x = 2$ with velocity $b(x, t) = 3$, its spatial derivative is $f'(2) = 4$.

The measurement changes at rate $(\mathscr{L}_t f)(2) = 3 \times 4 = 12$.

Let's calculate a generator with numbers

Let the measurement be $f(x) = x^2$. At state $x = 2$ with velocity $b(x, t) = 3$, its spatial derivative is $f'(2) = 4$.

The measurement changes at rate $(\mathscr{L}_t f)(2) = 3 \times 4 = 12$. Over a small interval of 0.01, its change is approximately 0.12.

Compose a function with the flow map

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t}f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\,T_{t,u}}$$

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t} f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\, T_{t,u}}$$

The transported measurement $(T_{s,t}f)(x)$ is $f(X_{s,t}(x))$, the value of f at the destination.

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t} f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\, T_{t,u}}$$

The transported measurement $(T_{s,t}f)(x)$ is $f(X_{s,t}(x))$, the value of f at the destination. Applying $T_{t,u}$ and then $T_{s,t}$ gives the same result as $T_{s,u}$.

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t}f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\,T_{t,u}}$$

The transported measurement $(T_{s,t}f)(x)$ is $f(X_{s,t}(x))$, the value of f at the destination. Applying $T_{t,u}$ and then $T_{s,t}$ gives the same result as $T_{s,u}$.

The cocycle on maps becomes the composition law on operators.

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t}f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\,T_{t,u}}$$

The transported measurement $(T_{s,t}f)(x)$ is $f(X_{s,t}(x))$, the value of f at the destination. Applying $T_{t,u}$ and then $T_{s,t}$ gives the same result as $T_{s,u}$.

The cocycle on maps becomes the composition law on operators.

The adjoint is the corresponding operator on densities.

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t}f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\,T_{t,u}}$$

The transported measurement $(T_{s,t}f)(x)$ is $f(X_{s,t}(x))$, the value of f at the destination. Applying $T_{t,u}$ and then $T_{s,t}$ gives the same result as $T_{s,u}$.

The cocycle on maps becomes the composition law on operators.

The adjoint is the corresponding operator on densities. It describes how probability changes so that expectations agree with the evolving measurements.

Compose a function with the flow map

Evaluate f at a future state:

$$\boxed{(T_{s,t}f)(x) = f\big(X_{s,t}(x)\big), \qquad T_{s,u} = T_{s,t}\,T_{t,u}}$$

The transported measurement $(T_{s,t}f)(x)$ is $f(X_{s,t}(x))$, the value of f at the destination. Applying $T_{t,u}$ and then $T_{s,t}$ gives the same result as $T_{s,u}$.

The cocycle on maps becomes the composition law on operators.

Using this operator, we can compute a future measurement for each possible starting state.

Why do we also need an adjoint?

Why do we also need an adjoint?

We can compute a measurement’s expectation by moving every state forward, or by evolving the probability distribution and then averaging the measurement.

Why do we also need an adjoint?

We can compute a measurement’s expectation by moving every state forward, or by evolving the probability distribution and then averaging the measurement.

The adjoint evolves the distribution so these two calculations agree.

Why do we also need an adjoint?

We can compute a measurement’s expectation by moving every state forward, or by evolving the probability distribution and then averaging the measurement.

The adjoint evolves the distribution so these two calculations agree. This connects the sample trajectories to the continuity equation.

Generators for continuous flows and discrete jumps

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

The continuous rate $(\mathcal{L}_t f)(x)$ is $b(x, t)$ dotted with $\nabla f(x)$.

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

The continuous rate $(\mathscr{L}_t f)(x)$ is $b(x, t)$ dotted with $\nabla f(x)$. The discrete rate sums $u_t(y, x) f(y)$ over states y, using rate-matrix entries $u_t(y, x)$.

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

The continuous rate $(\mathscr{L}_t f)(x)$ is $b(x, t)$ dotted with $\nabla f(x)$. The discrete rate sums $u_t(y, x) f(y)$ over states y, using rate-matrix entries $u_t(y, x)$.

Both generators describe the rate of change of f.

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

The continuous rate $(\mathscr{L}_t f)(x)$ is $b(x, t)$ dotted with $\nabla f(x)$. The discrete rate sums $u_t(y, x) f(y)$ over states y, using rate-matrix entries $u_t(y, x)$.

Both generators describe the rate of change of a measurement f.

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

The continuous rate $(\mathscr{L}_t f)(x)$ is $b(x, t)$ dotted with $\nabla f(x)$. The discrete rate sums $u_t(y, x) f(y)$ over states y, using rate-matrix entries $u_t(y, x)$.

Both generators describe the rate of change of a measurement f. Their adjoints evolve probability through the continuity equation or the discrete master equation.

Generators for continuous flows and discrete jumps

$$\text{continuous: } (\mathcal{L}_t f)(x) = b(x,t) \cdot \nabla f(x), \qquad \text{discrete: } (\mathcal{L}_t f)(x) = \sum_y u_t(y,x)\, f(y)$$

The continuous rate $(\mathscr{L}_t f)(x)$ is $b(x, t)$ dotted with $\nabla f(x)$. The discrete rate sums $u_t(y, x) f(y)$ over states y, using rate-matrix entries $u_t(y, x)$.

Both generators describe the rate of change of a measurement f. Their adjoints evolve probability through the continuity equation or the discrete master equation.

For diffusion, we also include a second spatial derivative in the continuous generator.

Velocity, flow map, and generator

Velocity, flow map, and generator

A velocity field b, which we train by flow matching.

Velocity, flow map, and generator

A velocity field b, which we train by flow matching.

A flow map $X_{s,t}$, which satisfies identity, cocycle and tangent.

Velocity, flow map, and generator

A velocity field b, which we train by flow matching.

A flow map $X_{s,t}$, which satisfies identity, cocycle and tangent.

A generator $\mathscr{L}_t$ for changes in measurements, with an adjoint for changes in density.

Velocity, flow map, and generator

A velocity field b, which we train by flow matching.

A flow map $X_{s,t}$, which satisfies identity, cocycle and tangent.

A generator $\mathscr{L}_t$ for changes in measurements, with an adjoint for changes in density.

In every method here, we learn the flow map and keep its relationship to the velocity.

Let's compute a flow map and check the axioms

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

The velocity is the Gaussian example we just solved

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

The velocity is the Gaussian example we just solved

- It is a translation plus a state-dependent expansion.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Integrating it from s to t is the flow map

- It is a translation plus a state-dependent expansion.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Integrating it from s to t is the flow map

- It is a translation plus a state-dependent expansion.
- We use fourth-order Runge-Kutta, so the discretization error is far below the effects we are checking.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Integrating over the full interval

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Integrating over the full interval

- At x = 0.7 this returns 4.1, which is 2 + 3x.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Two successive intervals give the same endpoint

- At x = 0.7 this returns 4.1, which is 2 + 3x.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Running it backward returns the starting point

- At x = 0.7 this returns 4.1, which is 2 + 3x.

```
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Differentiate in the starting point to get the Jacobian

- At x = 0.7 this returns 4.1, which is 2 + 3x.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Differentiate in the starting point to get the Jacobian

- At x = 0.7 this returns 4.1, which is 2 + 3x.
- The Jacobian is 3.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Differentiate in the starting point to get the Jacobian

- At x = 0.7 this returns 4.1, which is 2 + 3x.
- The Jacobian is 3. Its logarithm, log 3, equals the integral of the divergence.

```python
import torch
from torch.func import jacrev

def b(x, t):  # velocity field
    return 2.0 + 2.0 * (x - 2.0 * t) / (1.0 + 2.0 * t)

def flow(x, s, t, n=200):  # X_{s,t}(x) by RK4
    h = (t - s) / n
    for k in range(n):
        u = s + k * h
        k1 = b(x, u)
        k2 = b(x + 0.5 * h * k1, u + 0.5 * h)
        k3 = b(x + 0.5 * h * k2, u + 0.5 * h)
        k4 = b(x + h * k3, u + h)
        x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
    return x

x = torch.tensor(0.7)
print(flow(x, 0.0, 1.0))  # 2 + 3x = 4.1
print(flow(flow(x, 0.0, 0.5), 0.5, 1.0))  # the cocycle
print(flow(flow(x, 0.0, 1.0), 1.0, 0.0))  # the inverse
print(jacrev(lambda z: flow(z, 0.0, 1.0))(x))  # det J = 3
```

Could we learn the destination over a whole interval while preserving the rules of the flow?

Flow Map Matching

TMLR 2025

Flow map matching with stochastic interpolants: A mathematical framework for consistency models

Nicholas M. Boffi* *boffi@cims.nyu.edu*
Courant Institute of Mathematical Sciences

Michael S. Albergo* *albergo@nyu.edu*
Courant Institute of Mathematical Sciences

Eric Vanden-Eijnden *eve2@cims.nyu.edu*
Courant Institute of Mathematical Sciences

Reviewed on OpenReview: *https://openreview.net/forum?id=cqDH0e6ak2*

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

Let's write the map as a step of the right length

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

The flow map $X_{s,t}(x)$ gives the destination at time t.

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

The flow map $X_{s,t}(x)$ gives the destination at time t. It equals the starting state x plus the elapsed time t – s multiplied by the predicted average velocity $v_{s,t}(x)$.

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

The flow map $X_{s,t}(x)$ gives the destination at time t. It equals the starting state x plus the elapsed time t – s multiplied by the predicted average velocity $v_{s,t}(x)$.

At t = s, the elapsed time t – s is zero, so the map returns its input x.

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

The flow map $X_{s,t}(x)$ gives the destination at time t. It equals the starting state x plus the elapsed time t – s multiplied by the predicted average velocity $v_{s,t}(x)$.

At t = s, the elapsed time t – s is zero, so the map returns its input x.

The average velocity $v_{s,t}(x)$ is the displacement $X_{s,t}(x)$ – x divided by the elapsed time t – s.

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

The flow map $X_{s,t}(x)$ gives the destination at time t. It equals the starting state x plus the elapsed time t – s multiplied by the predicted average velocity $v_{s,t}(x)$.

At t = s, the elapsed time t – s is zero, so the map returns its input x.

The average velocity $v_{s,t}(x)$ is the displacement $X_{s,t}(x)$ – x divided by the elapsed time t – s.

We learn the same average velocity v with each method.

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)$$

The flow map $X_{s,t}(x)$ gives the destination at time t. It equals the starting state x plus the elapsed time t – s multiplied by the predicted average velocity $v_{s,t}(x)$.

At t = s, the elapsed time t – s is zero, so the map returns its input x.

The average velocity $v_{s,t}(x)$ is the displacement $X_{s,t}(x)$ – x divided by the elapsed time t – s.

We learn the same average velocity v with each method. The methods differ in their training loss.

Let's write the map as a step of the right length

Let the network produce the average velocity v:

$$\boxed{X_{s,t}(x) = x + (t - s)\, v_{s,t}(x)}$$

The flow map $X_{s,t}(x)$ gives the destination at time t. It equals the starting state x plus the elapsed time t – s multiplied by the predicted average velocity $v_{s,t}(x)$.

At t = s, the elapsed time t – s is zero, so the map returns its input x.

The average velocity $v_{s,t}(x)$ is the displacement $X_{s,t}(x)$ – x divided by the elapsed time t – s.

A long-step prediction must account for how the velocity changes along the whole route.

Now let the two times meet

Now let the two times meet

$$\boxed{v_{t,t}(x) = \lim_{s \to t} \frac{X_{s,t}(x) - x}{t - s} = b(x, t)}$$

Now let the two times meet

$$\boxed{v_{t,t}(x) = \lim_{s \to t} \frac{X_{s,t}(x) - x}{t - s} = b(x, t)}$$

The average velocity $v_{t,t}(x)$ is the limit of the displacement $X_{s,t}(x) - x$ divided by the elapsed time $t - s$ as s approaches t.

Now let the two times meet

$$v_{t,t}(x) = \lim_{s \to t} \frac{X_{s,t}(x) - x}{t - s} = b(x, t)$$

The average velocity $v_{t,t}(x)$ is the limit of the displacement $X_{s,t}(x) - x$ divided by the elapsed time $t - s$ as s approaches t. It equals the instantaneous velocity $b(x, t)$.

Now let the two times meet

$$v_{t,t}(x) = \lim_{s \to t} \frac{X_{s,t}(x) - x}{t - s} = b(x, t)$$

The average velocity $v_{t,t}(x)$ is the limit of the displacement $X_{s,t}(x) - x$ divided by the elapsed time $t - s$ as s approaches t. It equals the instantaneous velocity $b(x, t)$.

Flow matching trains the diagonal velocity $v_{t,t}(x)$ to equal the local velocity $b(x, t)$.

Now let the two times meet

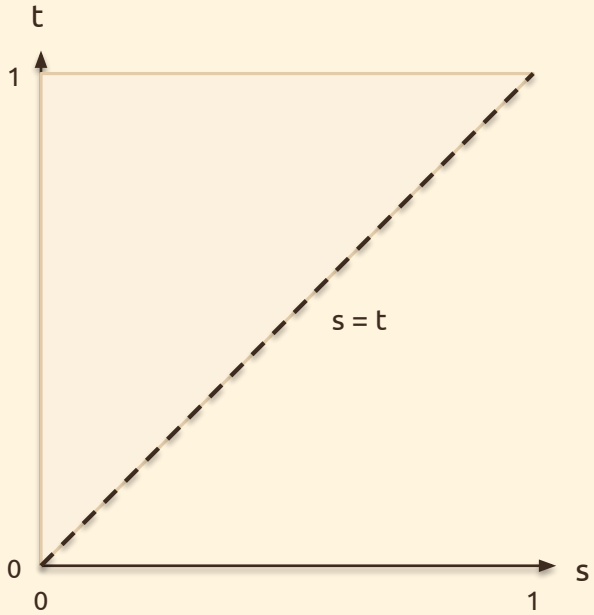
$$v_{t,t}(x) = \lim_{s \to t} \frac{X_{s,t}(x) - x}{t - s} = b(x, t)$$

The average velocity $v_{t,t}(x)$ is the limit of the displacement $X_{s,t}(x) – x$ divided by the elapsed time $t – s$ as s approaches t. It equals the instantaneous velocity $b(x, t)$.

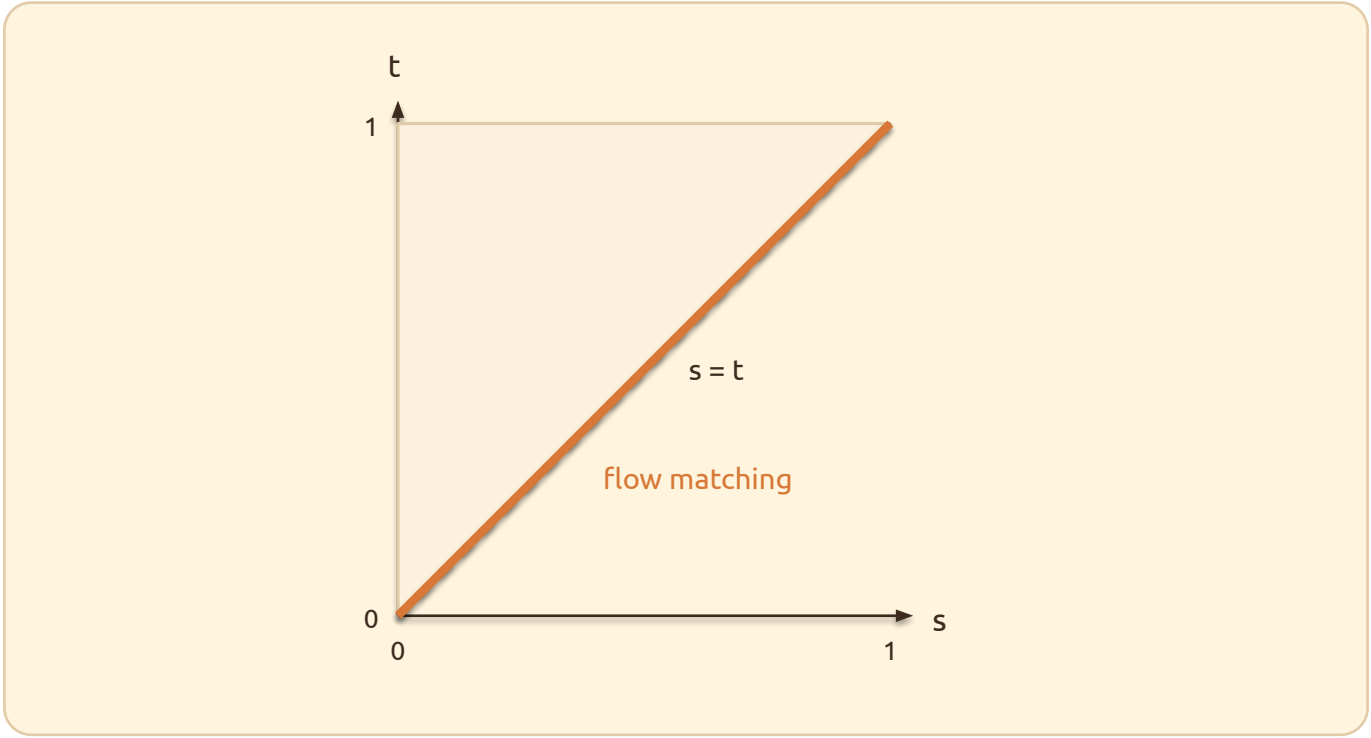
Flow matching trains the diagonal velocity $v_{t,t}(x)$ to equal the local velocity $b(x, t)$.

Training on the diagonal therefore learns the local velocity inside the two-time flow map.

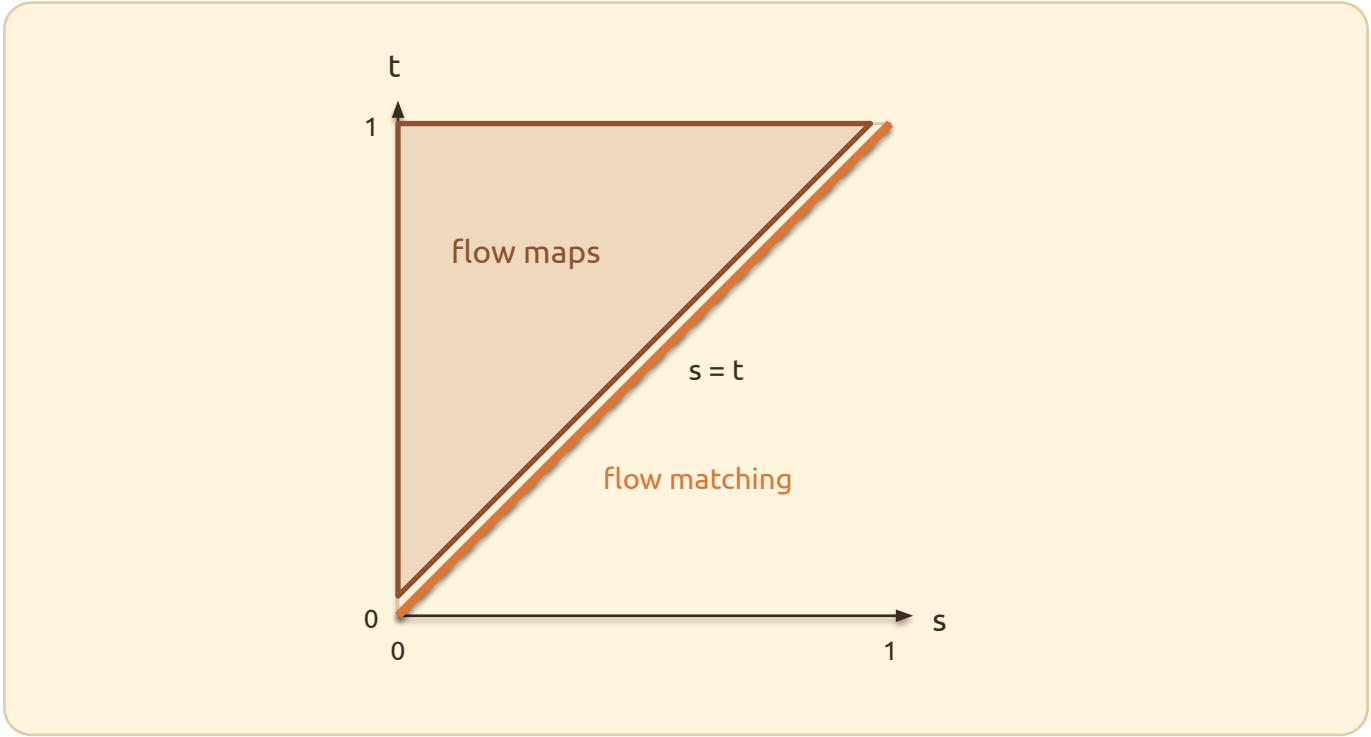
Let's draw the two times as a triangle



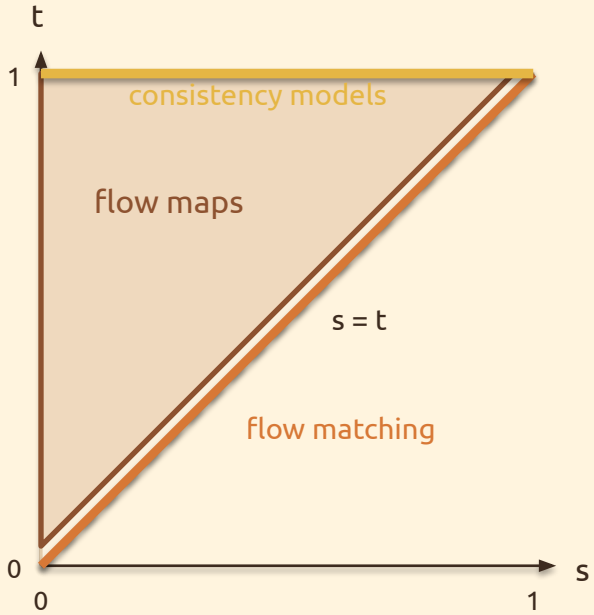
A velocity model has one time, so we train it on the diagonal



A flow map has a start and an end, so we train it on the whole triangle



Fixing the end time at 1 leaves one free time, which is a consistency model



How does the endpoint change with the arrival time?

How does the endpoint change with the arrival time?

Hold the start fixed and move the end:

$$\boxed{\frac{\partial}{\partial t} X_{s,t}(x) = b\big(X_{s,t}(x),\ t\big)}$$

How does the endpoint change with the arrival time?

Hold the start fixed and move the end:

$$\frac{\partial}{\partial t} X_{s,t}(x) = b\big(X_{s,t}(x),\ t\big)$$

The end-time derivative $\partial_t X_{s,t}(x)$ equals the instantaneous velocity $b(X_{s,t}(x), t)$ at the destination $X_{s,t}(x)$.

How does the endpoint change with the arrival time?

Hold the start fixed and move the end:

$$\frac{\partial}{\partial t} X_{s,t}(x) = b\big(X_{s,t}(x),\ t\big)$$

The end-time derivative $\partial_t X_{s,t}(x)$ equals the instantaneous velocity $b(X_{s,t}(x), t)$ at the destination $X_{s,t}(x)$.

As we move the end time, the endpoint moves along the trajectory at its local velocity.

How does the endpoint change with the arrival time?

Hold the start fixed and move the end:

$$\boxed{\frac{\partial}{\partial t} X_{s,t}(x) = b\big(X_{s,t}(x),\ t\big)}$$

The end-time derivative $\partial_t X_{s,t}(x)$ equals the instantaneous velocity $b(X_{s,t}(x), t)$ at the destination $X_{s,t}(x)$.

As we move the end time, the endpoint moves along the trajectory at its local velocity.

In a Lagrangian description, we follow one particle along its path.

How does the endpoint change with the arrival time?

Hold the start fixed and move the end:

$$\boxed{\frac{\partial}{\partial t} X_{s,t}(x) = b\big(X_{s,t}(x),\ t\big)}$$

The end-time derivative $\partial_t X_{s,t}(x)$ equals the instantaneous velocity $b(X_{s,t}(x), t)$ at the destination $X_{s,t}(x)$.

As we move the end time, the endpoint moves along the trajectory at its local velocity.

In this Lagrangian description, we follow the moving endpoint.

How does the endpoint change with the arrival time?

Hold the start fixed and move the end:

$$\frac{\partial}{\partial t} X_{s,t}(x) = b\big(X_{s,t}(x),\ t\big)$$

The end-time derivative $\partial_t X_{s,t}(x)$ equals the instantaneous velocity $b(X_{s,t}(x), t)$ at the destination $X_{s,t}(x)$.

As we move the end time, the endpoint moves along the trajectory at its local velocity.

In this Lagrangian description, we follow the moving endpoint. We change its arrival time t while keeping the starting state fixed.

Now differentiate in the start time

Now differentiate in the start time

Fix an arrival time t and follow one trajectory.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

The predicted destination $X_{s,t}(x_s)$ stays constant as the starting state x_s moves along its trajectory.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

The predicted destination $X_{s,t}(x_s)$ stays constant as the starting state x_s moves along its trajectory. Its total derivative $dX_{s,t}(x_s)/ds$ is therefore zero.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

The start-time change $\partial_s X_{s,t}(x)$ plus the input-motion term $b(x, s) \cdot \nabla_x X_{s,t}(x)$ is zero.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

The start-time change $\partial_s X_{s,t}(x)$ plus the input-motion term $b(x, s) \cdot \nabla_x X_{s,t}(x)$ is zero. Here $b(x, s)$ is the starting velocity and $\nabla_x X_{s,t}(x)$ is the map's spatial Jacobian.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

The start-time change $\partial_s X_{s,t}(x)$ plus the input-motion term $b(x, s)\cdot\nabla_x X_{s,t}(x)$ is zero. Here $b(x, s)$ is the starting velocity and $\nabla_x X_{s,t}(x)$ is the map's spatial Jacobian.

In an Eulerian description, we describe motion at fixed locations in space.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

The start-time change $\partial_s X_{s,t}(x)$ plus the input-motion term $b(x, s)\cdot\nabla_x X_{s,t}(x)$ is zero. Here $b(x, s)$ is the starting velocity and $\nabla_x X_{s,t}(x)$ is the map's spatial Jacobian.

In this Eulerian description, we vary the start time s and account for motion through the input x.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

The start-time change $\partial_s X_{s,t}(x)$ plus the input-motion term $b(x, s)\cdot\nabla_x X_{s,t}(x)$ is zero. Here $b(x, s)$ is the starting velocity and $\nabla_x X_{s,t}(x)$ is the map's spatial Jacobian.

In this Eulerian description, we vary the start time s and account for motion through the input x. These two changes cancel when the destination is fixed.

Now differentiate in the start time

Fix an arrival time t and follow one trajectory. Starting farther along that same trajectory should give the same destination.

So the total derivative in s vanishes: $$\frac{d}{ds} X_{s,t}(x_s) = 0 \quad \text{along a trajectory}$$

Expand that derivative by the chain rule: $$\boxed{\partial_s X_{s,t}(x) + b(x,s) \cdot \nabla_x X_{s,t}(x) = 0}$$

The start-time change $\partial_s X_{s,t}(x)$ plus the input-motion term $b(x, s) \cdot \nabla_x X_{s,t}(x)$ is zero. Here $b(x, s)$ is the starting velocity and $\nabla_x X_{s,t}(x)$ is the map's spatial Jacobian.

Moving the starting point along its trajectory should leave the prediction for a fixed arrival time unchanged.

Let's check the fixed destination with numbers

Let's check the fixed destination with numbers

At constant velocity 2, starting at x = 1 at time s = 0 gives destination 3 at time t = 1.

Let's check the fixed destination with numbers

At constant velocity 2, starting at x = 1 at time s = 0 gives destination 3 at time t = 1.

At time s = 0.25, the same trajectory is at x = 1.5.

Let's check the fixed destination with numbers

At constant velocity 2, starting at $x = 1$ at time $s = 0$ gives destination 3 at time $t = 1$.

At time $s = 0.25$, the same trajectory is at $x = 1.5$. The remaining displacement is $2 \times 0.75 = 1.5$, so the destination is still 3.

We now have three conditions to train on

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

Eulerian: differentiate in the start time and apply the spatial gradient to b at the input.

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

Eulerian: differentiate in the start time and apply the spatial gradient to b at the input.

Composition: require two shorter maps to agree with the map over the full interval.

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

Eulerian: differentiate in the start time and apply the spatial gradient to b at the input.

Composition: require two shorter maps to agree with the map over the full interval.

The Lagrangian and Eulerian conditions require a network derivative.

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

Eulerian: differentiate in the start time and apply the spatial gradient to b at the input.

Composition: require two shorter maps to agree with the map over the full interval.

The Lagrangian and Eulerian conditions require a network derivative. The composition condition compares finite steps using three forward passes.

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

Eulerian: differentiate in the start time and apply the spatial gradient to b at the input.

Composition: require two shorter maps to agree with the map over the full interval.

The Lagrangian and Eulerian conditions require a network derivative. The composition condition compares finite steps using three forward passes.

When v is continuous in both times, all three conditions have the same unique solution.

We now have three conditions to train on

Lagrangian: differentiate in the end time, evaluate b at the output.

Eulerian: differentiate in the start time and apply the spatial gradient to b at the input.

Composition: require two shorter maps to agree with the map over the full interval.

The Lagrangian and Eulerian conditions require a network derivative. The composition condition compares finite steps using three forward passes.

When v is continuous in both times, all three conditions have the same unique solution. They can behave differently with an imperfect network.

Let's check flow-map composition with numbers

Let's check flow-map composition with numbers

Use a constant velocity of 2.

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

$$X_{0.2,0.7}(1) = 1 + (0.7 - 0.2)(2) = 2$$

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

$$X_{0.2,0.7}(1) = 1 + (0.7 - 0.2)(2) = 2$$

The direct map moves one unit over the half-unit time interval.

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

$$X_{0.2,0.7}(1) = 1 + (0.7 - 0.2)(2) = 2$$

The direct map moves one unit over the half-unit time interval.

$$X_{0.4,0.7}(X_{0.2,0.4}(1)) = X_{0.4,0.7}(1.4) = 1.4 + 0.3(2) = 2$$

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

$$X_{0.2,0.7}(1) = 1 + (0.7 - 0.2)(2) = 2$$

The direct map moves one unit over the half-unit time interval.

$$X_{0.4,0.7}(X_{0.2,0.4}(1)) = X_{0.4,0.7}(1.4) = 1.4 + 0.3(2) = 2$$

The first short step reaches 1.4.

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

$$X_{0.2,0.7}(1) = 1 + (0.7 - 0.2)(2) = 2$$

The direct map moves one unit over the half-unit time interval.

$$X_{0.4,0.7}(X_{0.2,0.4}(1)) = X_{0.4,0.7}(1.4) = 1.4 + 0.3(2) = 2$$

The first short step reaches 1.4. The second reaches 2, matching the direct map.

Let's check flow-map composition with numbers

Use a constant velocity of 2. Start at position 1 at time 0.2 and finish at time 0.7, with an intermediate time of 0.4.

$$X_{0.2,0.7}(1) = 1 + (0.7 - 0.2)(2) = 2$$

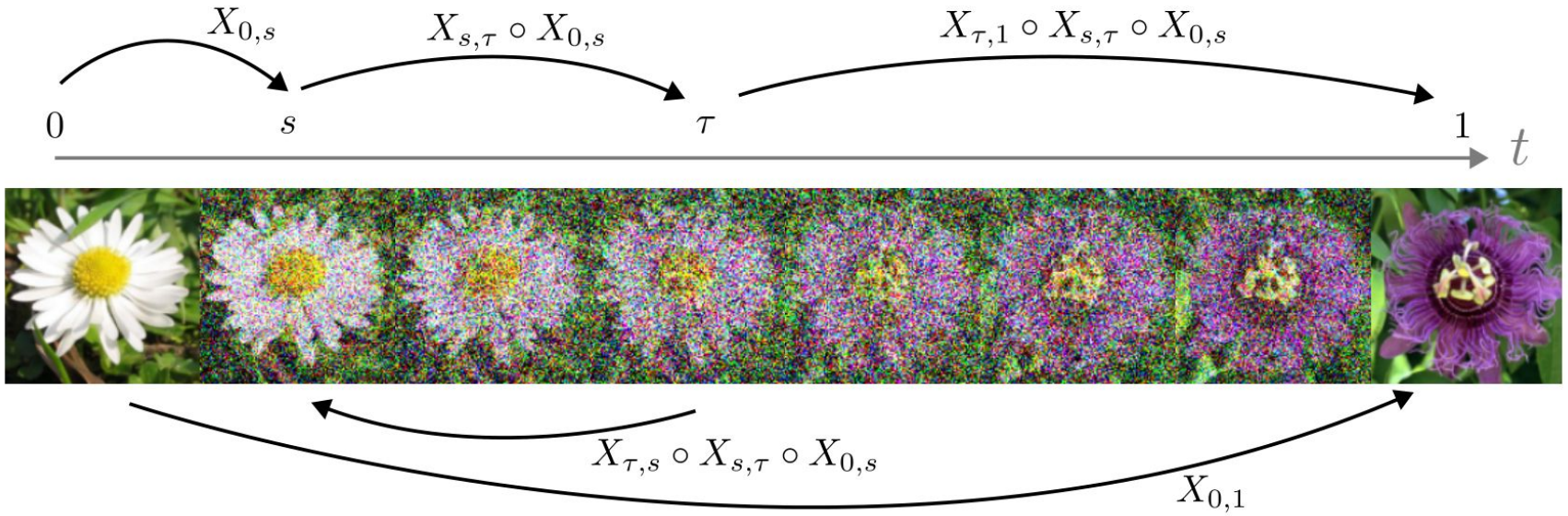
The direct map moves one unit over the half-unit time interval.

$$X_{0.4,0.7}(X_{0.2,0.4}(1)) = X_{0.4,0.7}(1.4) = 1.4 + 0.3(2) = 2$$

The first short step reaches 1.4. The second reaches 2, matching the direct map.

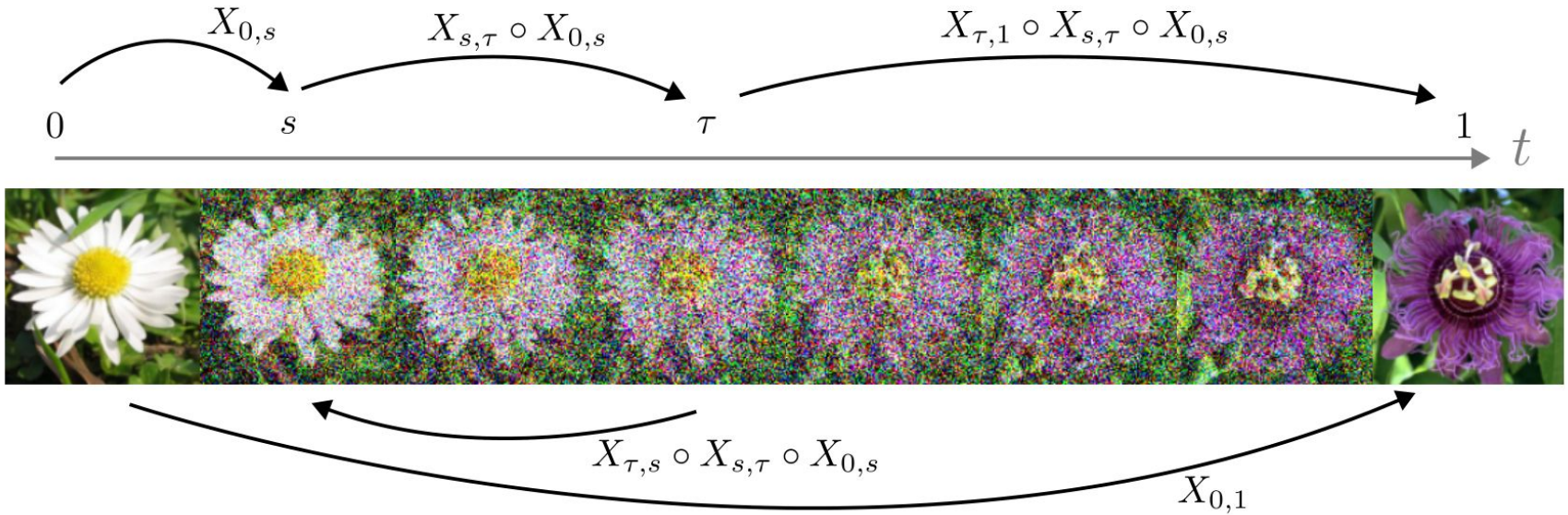
Under the composition condition, we reach the same final state regardless of where we split the interval.

Three training conditions for a two-time flow map



Boffi,..,Vanden-Eijnden. *TMLR* (2025)

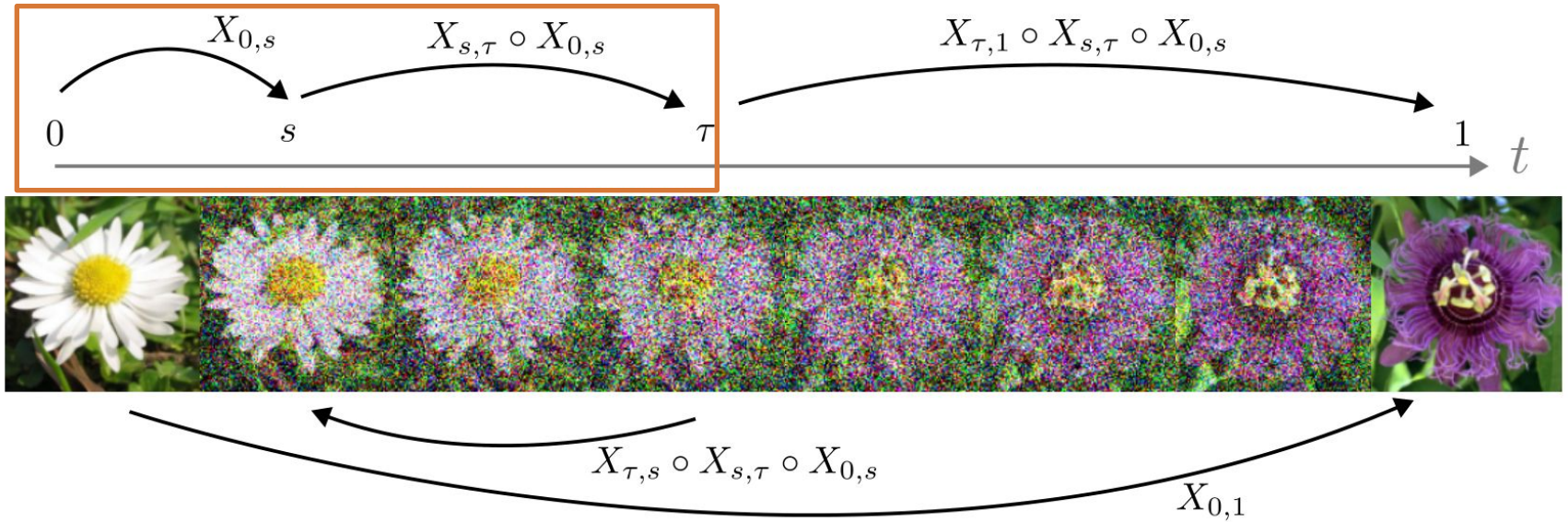
Three training conditions for a two-time flow map



A state goes from one time to another in a single flow map evaluation.

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

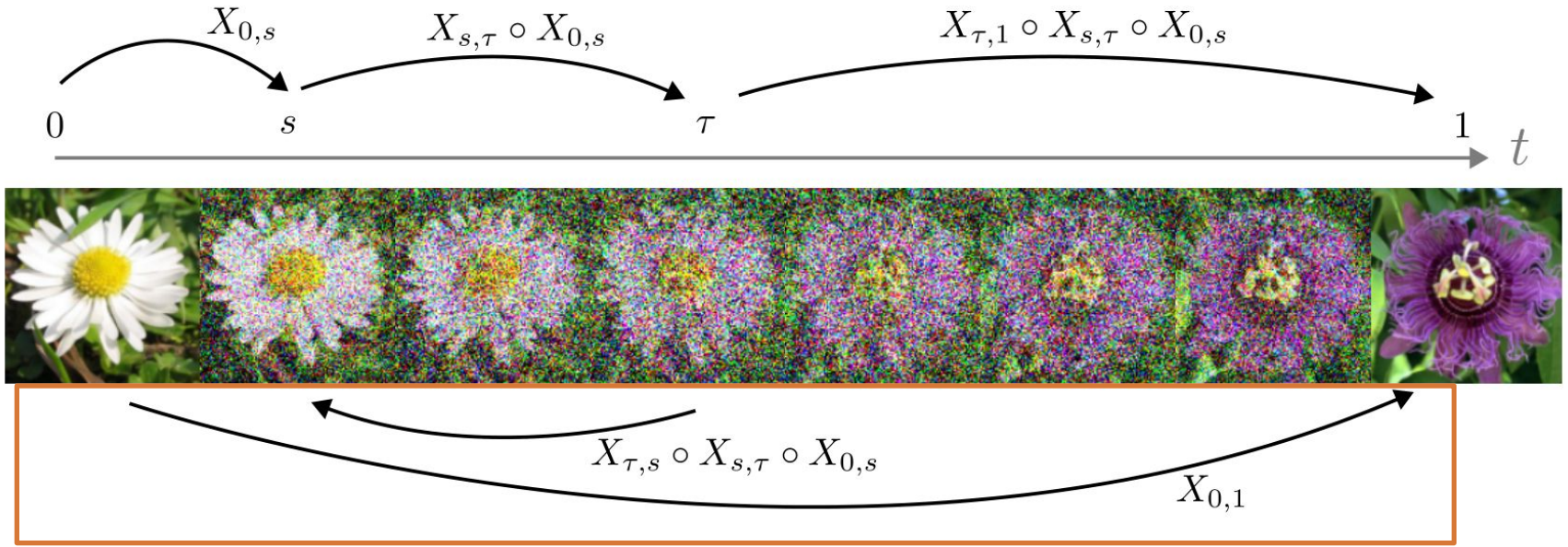
A flow map predicts the endpoint of any chosen time interval



A state goes from one time to another in a single flow map evaluation.

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

Two consecutive maps should reach the same final state as one long map



A state goes from one time to another in a single flow map evaluation.

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

On CIFAR-10, compare distance to data with distance to the teacher

Method	N=2		N=4		Baseline
	FID	T-FID	FID	T-FID	
SI	112.42	-	34.84	-	5.53
EMD	48.32	34.19	44.35	30.74	5.53
LMD	7.13	1.27	6.04	1.05	5.53
PFMM	18.35	7.02	11.14	1.52	8.44

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

N is the sampling-step count; FID compares with data and T-FID with the teacher

Method	N=2		N=4		Baseline
	FID	T-FID	FID	T-FID	
SI	112.42	-	34.84	-	5.53
EMD	48.32	34.19	44.35	30.74	5.53
LMD	7.13	1.27	6.04	1.05	5.53
PFMM	18.35	7.02	11.14	1.52	8.44

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

The teacher's baseline FID is 5.53; lower FID means closer image distributions

Method	N=2		N=4		Baseline
	FID	T-FID	FID	T-FID	
SI	112.42	-	34.84	-	5.53
EMD	48.32	34.19	44.35	30.74	5.53
LMD	7.13	1.27	6.04	1.05	5.53
PFMM	18.35	7.02	11.14	1.52	8.44

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

At four steps, Lagrangian distillation has FID 6.04 and teacher-FID 1.05

Method	N=2		N=4		Baseline
	FID	T-FID	FID	T-FID	
SI	112.42	-	34.84	-	5.53
EMD	48.32	34.19	44.35	30.74	5.53
LMD	7.13	1.27	6.04	1.05	5.53
PFMM	18.35	7.02	11.14	1.52	8.44

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

Eulerian distillation has teacher-FID 30.74 with the same teacher and architecture

Method	N=2		N=4		Baseline
	FID	T-FID	FID	T-FID	
SI	112.42	-	34.84	-	5.53
EMD	48.32	34.19	44.35	30.74	5.53
LMD	7.13	1.27	6.04	1.05	5.53
PFMM	18.35	7.02	11.14	1.52	8.44

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

Composition reaches teacher-FID 1.52 using a different teacher with baseline FID 8.44

Method	N=2		N=4		Baseline
	FID	T-FID	FID	T-FID	
SI	112.42	-	34.84	-	5.53
EMD	48.32	34.19	44.35	30.74	5.53
LMD	7.13	1.27	6.04	1.05	5.53
PFMM	18.35	7.02	11.14	1.52	8.44

Boffi,..,Vanden-Eijnden. *TMLR* (2025)

Could the model learn its own long steps while learning the local velocity too?